

# API Documentation

Rotana Spa Management System — HTTP API reference.

## 1. Conventions

- **Base URL:** `http://<host>/api`
- **Auth:** `Authorization: Bearer <token>` on all endpoints except `/api/login*`.
- **Content-Type:** `application/json` for requests and responses.
- **Success status:** `200 OK` (read/update), `201 Created` (create).
- **Error status:** `400 bad request`, `401 unauthorised`, `403 permission denied`, `404 not found`, `500 server error`, `503 tables not installed`.
- **Error body:** `{ "error": "<message>" }`
- **Tenancy:** non-super-admin users are always scoped to their own `company_id`. Super admins must pass `company_id` where noted.

## 2. Authentication

`POST /api/login`

Authenticates a tenant user.

```
{ "email": "admin@rotanaspa.com", "password": "admin123" }
```

```
{ "token": "<jwt>", "user": { "id": 1, "name": "Admin", "email": "...", "role": "admin", "company_id": 1, "compan
```

Entry variants:

- `POST /api/login/super-admin`
- `POST /api/login/employee`
- `POST /api/login/guest`
- `POST /api/login/company`

## 3. Membership

### Plans

| Method | Path                      | Body / notes                                                                   |
|--------|---------------------------|--------------------------------------------------------------------------------|
| GET    | /api/membership/plans     | list                                                                           |
| POST   | /api/membership/plans     | { name, type (gym spa cafe general), description, duration_days, max_members } |
| GET    | /api/membership/plans/:id | detail (includes member_count)                                                 |
| PUT    | /api/membership/plans/:id | partial: { name, type, description, duration_days, max_members, is_active }    |
| DELETE | /api/membership/plans/:id |                                                                                |

## Members

| Method | Path                            | Body / notes                                                                                                          |
|--------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| GET    | /api/membership/members         | list                                                                                                                  |
| POST   | /api/membership/members         | { full_name, phone, email, id_number, address, plan_id, start_date, end_date?, notes } ; customer_id auto ( MEM-### ) |
| GET    | /api/membership/members/:id     | detail                                                                                                                |
| PUT    | /api/membership/members/:id     | update                                                                                                                |
| DELETE | /api/membership/members/:id     |                                                                                                                       |
| GET    | /api/membership/members/next-id | next customer code                                                                                                    |

## Subscriptions

| Method | Path                          | Body                                                                        |
|--------|-------------------------------|-----------------------------------------------------------------------------|
| GET    | /api/membership/subscriptions | list                                                                        |
| POST   | /api/membership/subscriptions | { member_id, plan_id?, start_date?, end_date, billing_cycle?, auto_renew? } |
| PUT    | /api/membership/subscriptions | { id, plan_id?, end_date?, billing_cycle?, status?, auto_renew? }           |

## Payments

- GET/POST /api/membership/payments — { member\_id, amount, currency, payment\_method, reference, payment\_date, notes } .

## Facilities / gates / rate-cards / rfid / qr / day-tickets / sessions

Each is a standard resource with GET (list) + POST + PUT + DELETE :

| Path                        | POST body (key fields)                                                                                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| /api/membership/facilities  | { name, type (room zone pool sauna steam cafe gym changing other), capacity, description, is_active }                               |
| /api/membership/gates       | { name, location, gate_type, reader_type, is_active }                                                                               |
| /api/membership/rate-cards  | { name, facility_id?, service_type (membership day_pass session facility service), price, currency?, duration_minutes?, is_active } |
| /api/membership/rfid-cards  | { card_uid, member_id?, type, status?, issued_date?, expiry_date? }                                                                 |
| /api/membership/qr-passes   | { member_id?, pass_type, token?, issued_date?, expiry_date?, max_uses? }                                                            |
| /api/membership/day-tickets | { guest_name?, facility_id?, rate_id?, price, currency? }                                                                           |
| /api/membership/sessions    | { member_id?, card_uid?, facility_id? }; PUT check-out                                                                              |

PUT handlers on plans/gates/facilities/rate-cards/subscriptions update **only provided fields** — omitted fields are preserved.

## Attendance

| Method   | Path                                 | Notes                                                                                                                |
|----------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| GET      | /api/membership/attendance           | today's attendance                                                                                                   |
| POST     | /api/membership/attendance/check-in  | { member_id or full_name + customer_id, card_uid? }                                                                  |
| POST     | /api/membership/attendance/check-out | { member_id or session_id }                                                                                          |
| GET      | /api/membership/attendance/history   | history                                                                                                              |
| GET      | /api/membership/access-logs          | gate access logs                                                                                                     |
| GET/POST | /api/membership/appointments         | { customer_name, customer_phone?, appointment_type, staff_id?, service_id?, start_time, end_time?, status?, notes? } |

## 4. Spa operations (visits & service orders)

### Visits

| Method | Path            | Notes                                |
|--------|-----------------|--------------------------------------|
| GET    | /api/spa/visits | query: ?status=&q=&company_id=&date= |

| Method | Path            | Notes                                                                                               |
|--------|-----------------|-----------------------------------------------------------------------------------------------------|
| POST   | /api/spa/visits | { customer_name, customer_phone?, member_id?, appointment_id?, notes? } → creates visit + SPA-##### |
| PUT    | /api/spa/visits | action-based state machine (below)                                                                  |

PUT /api/spa/visits actions ( { id, action, ... } ):

- assign → { id, action:"assign", therapist\_record\_id } (requires active therapist; checked\_in → assigned )
- start → { id, action:"start" } (requires assigned therapist; → in\_treatment )
- cancel → { id, action:"cancel" }
- handoff → { id, action:"handoff" } (requires finished; visit + order → handed\_to\_cashier )
- default → updates { id, notes?, customer\_phone? }

## Visit services

| Method | Path                         | Notes                                                                                        |
|--------|------------------------------|----------------------------------------------------------------------------------------------|
| GET    | /api/spa/visits/:id/services | list service lines                                                                           |
| POST   | /api/spa/visits/:id/services | { service_record_id, quantity? } — requires visit in_treatment ; upserts by (visit, service) |

## Service order (per visit)

| Method | Path                              | Notes                                                 |
|--------|-----------------------------------|-------------------------------------------------------|
| GET    | /api/spa/visits/:id/service-order | order for visit                                       |
| POST   | /api/spa/visits/:id/service-order | action state machine: finish, refresh, print, handoff |

- finish : validates in\_treatment visit + ≥1 service, marks visit finished , generates SO-SPA-##### draft order with a snapshot of lines.
- refresh : regenerates the snapshot (if still editable).
- print : sets printed\_at/printed\_by , increments print\_count , status → printed ; returns slip data for thermal printing.
- handoff : marks order handed\_to\_cashier .

## Service orders (list)

| Method | Path                    | Notes                                                                                                             |
|--------|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| GET    | /api/spa/service-orders | ?status=&company_id= → { orders: [], summary: { drafts, printed, handed_to_cashier, total_items }, capabilities } |

## 5. Dynamic workspaces

`/api/spa/:section/:module`

Generic CRUD over `spa_management_records`, validated against `lib/spa-modules.ts`.

| Method | Notes                                                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>?q=&amp;status=&amp;limit=&amp;offset=&amp;company_id=</code> → <code>{ records, filteredCount, summary, capabilities }</code> |
| POST   | module fields in body (or under <code>details</code> ); <code>company_id</code> required for super admin                             |
| PUT    | <code>{ id, ...fields, status? }</code>                                                                                              |
| DELETE | <code>{ id }</code> (soft delete via <code>deleted_at</code> )                                                                       |

Known modules (section/slug):

| Section    | Modules                                                                                                                                       |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| customers  | <code>medical-records</code> , <code>loyalty</code>                                                                                           |
| membership | <code>freeze-transfer</code>                                                                                                                  |
| operations | <code>queue</code> , <code>customer-requests</code> , <code>towel-management</code>                                                           |
| gym        | <code>trainers</code> , <code>workout-plans</code> , <code>fitness-assessments</code> , <code>body-measurements</code> , <code>classes</code> |
| spa        | <code>services</code> , <code>therapists</code> , <code>packages</code>                                                                       |
| inventory  | <code>products</code> , <code>consumables</code> , <code>stock-usage</code> , <code>suppliers</code>                                          |
| staff      | <code>employees</code> , <code>schedules</code> , <code>commission</code> , <code>performance</code>                                          |
| facilities | <code>lockers</code> , <code>equipment</code> , <code>maintenance</code>                                                                      |

## 6. Reports

GET `/api/spa/reports/:report`

Query: `?from=YYYY-MM-DD&to=YYYY-MM-DD&company_id=`

Reports: `membership`, `attendance`, `service-orders`, `therapist`, `trainer`, `inventory`.

Response:

```
{
  "summary": [{ "label": "Active Members", "value": 12, "format": "number" }],
  "columns": [{ "key": "plan_name", "label": "Plan", "format": "text" }],
  "rows": [ { ... } ],
  "range": { "from": "2026-07-01", "to": "2026-07-30" }
}
```

## 7. Administration

| Method              | Path                                      | Notes                                                               |
|---------------------|-------------------------------------------|---------------------------------------------------------------------|
| GET/POST            | /api/users                                | create { name, email, password, role, phone }                       |
| GET/PUT/DELETE      | /api/users/:id                            | update role/active                                                  |
| GET                 | /api/roles                                | role list                                                           |
| GET                 | /api/roles/:id/permissions                | permission matrix for role                                          |
| PUT                 | /api/roles/:id/permissions                | { permissions: { resource:<br>[view,create,edit,delete,approve] } } |
| GET/POST/PUT        | /api/settings                             | company settings (key/value)                                        |
| GET/POST/PUT/DELETE | /api/settings/id-definitions (+<br>/:id ) | numbering rules                                                     |
| GET                 | /api/audit-logs                           | audit trail                                                         |
| GET/POST            | /api/companies                            | company CRUD (+ /:id , /api/companies/public )                      |
| GET/POST            | /api/demo-licenses (+ /:id )              | licenses                                                            |
| GET                 | /api/modules                              | module catalogue                                                    |
| GET/POST/PUT/DELETE | /api/notifications (+ /:id )              | user notifications                                                  |
| GET                 | /api/dashboard/stats                      | dashboard KPIs                                                      |
| GET                 | /api/search                               | global search                                                       |
| GET                 | /api/admin/dashboard                      | platform dashboard                                                  |

## 8. Error reference

| Code               | Meaning                                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 42P01<br>(500/503) | Tables missing — apply db-migration-v33.sql / db-migration-v34.sql .                                                                   |
| 23505 (500)        | Unique violation (duplicate visit_no, order_no, card_uid...).                                                                          |
| 403                | Permission denied — role lacks resource/action .                                                                                       |
| 400                | Validation — e.g. Member and end date are required , Start the treatment before recording services used , Active therapist not found . |

## 9. Examples

## Create a visit and run the handoff flow

```
TOKEN=$(curl -s -X POST $BASE/api/login -H 'Content-Type: application/json' \
  -d '{"email":"admin@rotanaspa.com","password":"admin123"}' | jq -r .token)

# check-in
VISIT=$(curl -s -X POST $BASE/api/spa/visits -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"customer_name":"Ruth Desta","customer_phone":"+251911000002"}')

# assign therapist (use an existing spa/therapists record id)
curl -s -X PUT $BASE/api/spa/visits -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"id":$(echo $VISIT|jq .id), "action": "assign", "therapist_record_id": 4}'

# start treatment
curl -s -X PUT $BASE/api/spa/visits -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"id":$(echo $VISIT|jq .id), "action": "start"}'

# add a service
curl -s -X POST $BASE/api/spa/visits/$(echo $VISIT|jq .id)/services -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"service_record_id": 5, "quantity": 1}'

# finish -> draft order
curl -s -X POST $BASE/api/spa/visits/$(echo $VISIT|jq .id)/service-order -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"action": "finish"}'

# print
curl -s -X POST $BASE/api/spa/visits/$(echo $VISIT|jq .id)/service-order -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"action": "print"}'

# hand to cashier
curl -s -X POST $BASE/api/spa/visits/$(echo $VISIT|jq .id)/service-order -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"action": "handoff"}'
```

## Dynamic workspace — create a spa service

```
curl -s -X POST $BASE/api/spa/spa/services -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"service_name": "Moroccan Bath", "category": "body-treatment", "duration"
```

